FILE SYSTEM CLI DESIGN

Block Mapping, Inode Visibility & Supervisor Demo Guide

A clean command-line interface design for showing how files are stored inside disk.img, including inode numbers, single/multiple block mappings, bitmap state, and block reuse after deletion.

Project	File System
Student	Saadab Rahman Ridmi
Roll	BSSE1754
Supervisor	Dr. B M Mainul Hossain

1. CLI Objective

The CLI should not only allow normal file operations. It should expose the internal filesystem state so that a supervisor can directly see how a file is connected to its inode and to one or more physical blocks.

- Show which inode belongs to each file.
- Show the exact physical block number(s) used by a file.
- Show whether a file uses direct or indirect addressing.
- Show which blocks are free and which are occupied.
- Show that deleted blocks become free and can be reused.

2. Recommended CLI Layout

```
========================================
      FILE SYSTEM - CLI
========================================
Type 'help' to see commands.

fs>
```

3. Command Groups

Group	Commands	Purpose
Disk	format, info, bitmap, map	Initialize storage and inspect block usage.
Files	create, write, read, append, delete, stat, blocks	Create and inspect file data.
Directories	mkdir, ls, rmdir	Manage folders and directory contents.
Management	rename, copy, move, search	Perform higher-level file management.
Debug	inodes	Inspect the inode table.
System	help, exit	CLI control commands.

Suggested help output

```
fs> help

FILE SYSTEM COMMANDS

DISK
 format              Format virtual disk
 info                Show filesystem information
 bitmap              Show block bitmap
 map                 Show block ownership

FILES
 create <path>       Create file
 write <path> <data>   Write file
 read <path>         Read file
 append <path> <data>  Append data
 delete <path>       Delete file
 stat <path>         Show file information
 blocks <path>       Show file block mapping

DIRECTORIES
 mkdir <path>        Create directory
 ls [path]           List directory
 rmdir <path>        Remove empty directory

MANAGEMENT
 rename <src> <name>   Rename
 copy <src> <dst>      Copy
 move <src> <dst>      Move
 search <name>       Search

DEBUG
 inodes              Show inode table

SYSTEM
 help
 exit
```

4. Most Important Supervisor-Facing Commands

The four most important inspection commands are stat, blocks, bitmap, and map. Together they make the internal storage behavior visible.

4.1 stat <file>

Shows file metadata, inode number, size, and block references.

```
fs> stat report.txt


----------------------------------------
File Information
----------------------------------------
Name         : report.txt
Type         : FILE
Inode        : 5
Size         : 1200 bytes
Block Size   : 512 bytes
Blocks Used  : 3

Direct Blocks :
 [0] -> Block 20
 [1] -> Block 21
 [2] -> Block 22

Indirect Block: Not Used
----------------------------------------
```

Conceptual mapping:

```
report.txt
  |
  +--> inode 5
       |
       +--> block 20
       +--> block 21
       +--> block 22
```

4.2 blocks <file>

Shows logical file blocks and their physical disk block mapping.

```
fs> blocks report.txt

File: report.txt
Size: 1200 bytes

Block Mapping:

Logical Block     Physical Block
-------------     --------------
0                 20
1                 21
2                 22

Total Blocks: 3
```

A visual version can also be displayed:

```
report.txt
  |
  +--- Block 20   [bytes 0 - 511]
  |
  +--- Block 21   [bytes 512 - 1023]
  |
  +--- Block 22   [bytes 1024 - 1199]
```

5. Single-Block and Multi-Block File Demonstration

5.1 Single-block file

```
fs> create hello.txt
File created successfully.
Inode allocated: 3

fs> write hello.txt "Hello File System"
18 bytes written.
Block allocated: 15

fs> blocks hello.txt

File: hello.txt
Size: 18 bytes

Block Mapping:
Logical Block 0 -> Physical Block 15

Total Blocks: 1
```

5.2 Multi-block file

If BLOCK_SIZE is 512 bytes and a file contains 1300 bytes, the file requires 3 blocks.

```
ceil(1300 / 512) = 3 blocks

fs> write big.txt <1300 bytes of data>

Writing file...

Allocated Block 30
Allocated Block 31
Allocated Block 35

1300 bytes written successfully.
Blocks used: 3

fs> blocks big.txt

File: big.txt
Inode: 7
Size: 1300 bytes
Block Size: 512 bytes

Block Mapping:
[0] -> Block 30
[1] -> Block 31
[2] -> Block 35

Total Blocks Used: 3
```

The physical blocks do not need to be consecutive. Showing 30, 31, and 35 is useful because it proves the filesystem is performing block allocation rather than simply storing a file in one continuous area.

6. Enhanced ls Command

Instead of listing only names, ls can show type, inode, size, and block ownership.

```
fs> ls

NAME           TYPE      INODE     SIZE      BLOCKS
-------------------------------------------------------
hello.txt       FILE      3         18 B      15
report.txt      FILE      5         1200 B    20,21,22
documents       DIR       6         --        25
```

7. Whole-Disk Inspection Commands

7.1 map

Shows block ownership across the entire virtual disk.

```
fs> map

==========================================
        FILE SYSTEM BLOCK MAP
==========================================

Block    Status    Owner
------------------------------------------
0        USED      Superblock
1        USED      Block Bitmap
2        USED      Inode Table
3        USED      Inode Table
4        FREE      -
5        FREE      -

15       USED      hello.txt
20       USED      report.txt
21       USED      report.txt
22       USED      report.txt
23       FREE      -
24       FREE      -
25       USED      /documents
------------------------------------------
```

7.2 bitmap

Displays the free/used state of disk blocks.

```
fs> bitmap

Block Bitmap

Block:   0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Status:  1 1 1 1 0 0 0 0 0 0  0  0  0  0  0  1

1 = USED
0 = FREE
```

Alternative summary:

```
USED BLOCKS:
0 1 2 3 15 20 21 22 25

FREE BLOCKS:
4 5 6 7 8 9 10 11 12 13 14 16 17 18 ...
```

7.3 inodes

Displays the inode table and each inode's block references.

```
fs> inodes

INODE TABLE

INODE   TYPE    SIZE      BLOCKS
-------------------------------------------
0       DIR     --        10
3       FILE    18 B      15
5       FILE    1200 B    20,21,22
6       DIR     --        25
7       FILE    1300 B    30,31,35
```

8. Direct and Indirect Block Display

For larger files, stat should distinguish direct data blocks from an indirect pointer block.

```
fs> stat large.txt

Name         : large.txt
Inode        : 12
Size         : 7500 bytes

Direct Blocks:
[0] -> 60
[1] -> 61
[2] -> 65
[3] -> 70

Indirect Block:
Pointer Block -> 80

Indirect Data Blocks:
[0] -> 81
[1] -> 84
[2] -> 90
[3] -> 91
...
            inode 12
              |
       +---------+---------+
       |                   |
    Direct              Indirect
       |                   |
    60 61 65 70          Block 80
                           |
                  81 84 90 91 ...
```

9. Recommended Supervisor Demo Sequence

The following sequence is short but demonstrates file operations, inode management, block allocation, bitmap management, and block reuse.

```
fs> format

Filesystem formatted successfully.
Block Size   : 512 bytes
Total Blocks : 2048
Total Inodes : 128
```

```
fs> create report.txt
File created.
Inode: 1

fs> write report.txt <1200 bytes>

Allocated blocks:
20
21
22

1200 bytes written successfully.

fs> stat report.txt

Inode       : 1
Size        : 1200 bytes
Blocks Used : 3
Blocks      : 20, 21, 22

fs> blocks report.txt
# Show logical -> physical block mapping

fs> bitmap
# Show blocks 20, 21, and 22 as USED

fs> delete report.txt

Deleting report.txt...

Freed Block 20
Freed Block 21
Freed Block 22
Freed Inode 1

File deleted successfully.

fs> bitmap
# Show blocks 20, 21, and 22 as FREE again
```

10. What This Demo Proves

Feature	What the CLI Demonstrates
File Operations	A file can be created, written, inspected, and deleted.
Inode Management	Each file has an inode containing metadata and block references.
Block Allocation	Files receive one or more physical disk blocks.
Free-Space Bitmap	Used blocks become marked and are freed after deletion.
Block Reuse	Freed blocks become available for later allocations.
Direct / Indirect Addressing	Large files can extend beyond the direct block pointer list.
Directory / Path Layer	The same inspection commands can work with nested paths.

11. Implementation Priority

If time is limited, implement the inspection commands in this order:

1. blocks <file> - most direct proof of file-to-block mapping
2. stat <file> - shows inode, size, and block references
3. bitmap - shows free/used state
4. map - shows block ownership across the whole disk
5. inodes - useful debug view for inode management

12. Final Recommended CLI

```
DISK
  format
  info
  bitmap
  map

FILES
  create <path>
  write <path> <data>
  read <path>
  append <path> <data>
  delete <path>
  stat <path>
  blocks <path>

DIRECTORIES
  mkdir <path>
  ls [path]
  rmdir <path>

MANAGEMENT
  rename <src> <name>
  copy <src> <dst>
  move <src> <dst>
  search <name>

DEBUG
  inodes

SYSTEM
  help
  exit
```

Key supervisor-facing commands: blocks, stat, bitmap, map